

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Лабораторная работа №1
«Проектирование функциональной модели нейронной сети»
по дисциплине «Организация вычислительных систем»

Выполнил:

Студент группы Р4130

Дворкин Борис Александрович

Преподаватель:

Быковский Сергей Вячеславович

Санкт-Петербург

2026 г.

Содержание

1. Задание	3
2. Модель сети	3
Параметры модели	3
3. Реализация	5
3.1. Прямой проход	5
3.2. Ошибка	5
3.3. Обратный проход	6
3.4. Обучение	6
3.5. Классификация	7
4. Выборки	8
5. Результаты	10
5.1. Сравнение структур	10
5.2. Варьирование α	10
5.3. Длительность обучения	10
5.4. Характеристики выбранной сети	11
5.5. Результат на тесте по группам	11
6. Пример работы сети	12
7. Выводы	12

1. Задание

Разработать функциональную модель многослойной полносвязной сети прямого распространения с сигмоидальной функцией активации, распознающей круг, квадрат и треугольник на чёрно-белых изображениях 7×7. Обучение методом обратного распространения ошибки. Выбрать структуру с наилучшими обобщающими свойствами и оценить время обучения, число эпох, время вычисления выхода и требуемую память.

Реализация: C++17 без библиотек нейронных сетей. Сборка и запуск: `make run` (обучение и демонстрация), `make compare` (сравнение структур), `make alpha` (варьирование α), `make figures` (рисунки и таблицы отчёта из вывода программы).

2. Модель сети

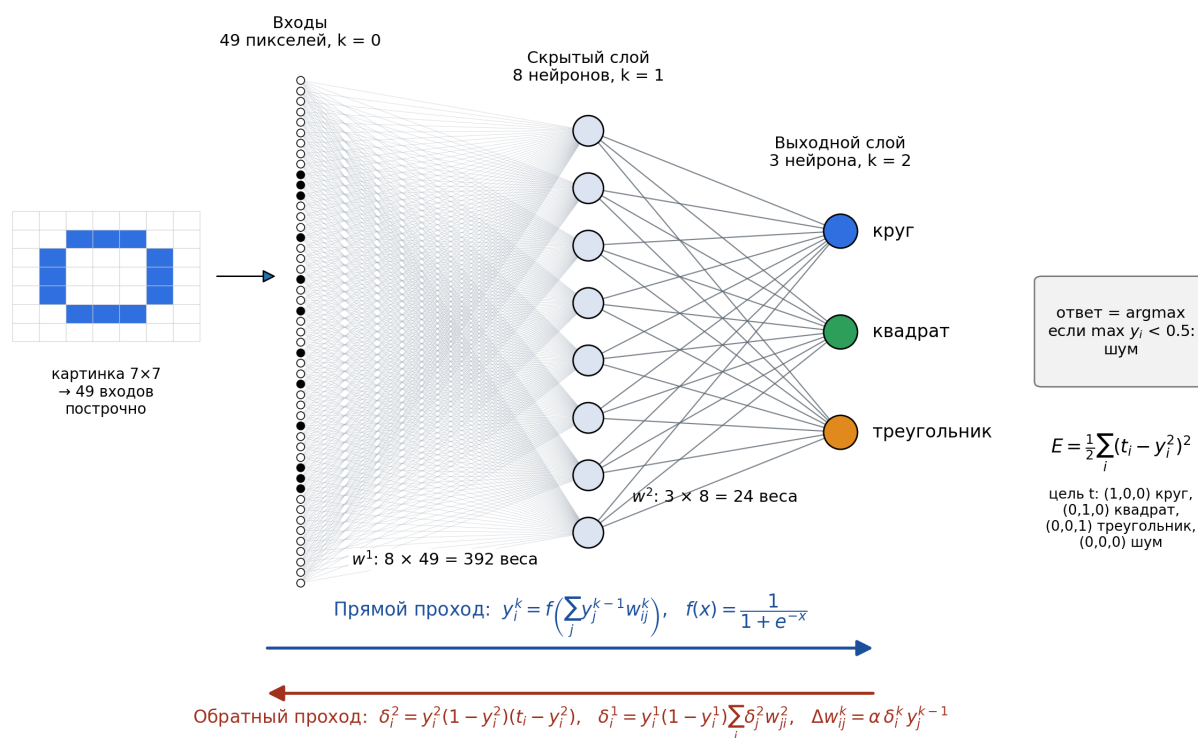


Рисунок 1. Сеть 49-8-3: слои, связи, формулы прямого и обратного прохода

Параметры модели

Параметр	Выбор	Причина
Активация	сигмоида $f(x) = 1 / (1 + e^{-x})$	Требование задания; выход в (0, 1); производная $f(1 - f)$ используется в δ

Параметр	Выбор	Причина
Ошибка	$E = \frac{1}{2} \sum (t_i - y_i)^2$	В методичке пропущен квадрат: без него ошибки разного знака гасят друг друга, а формулы δ выводятся только из E с квадратом
Выходы	3; шум: цель (0, 0, 0), ответ «шум», если $\max y_i < 0.5$	Задание требует ровно три выхода
Bias	нет	Строго по формуле y_i^k из задания
Порог eps	0.1	Значение из методички; при eps = 0.1 порог 0.5 верно классифицирует всю обучающую выборку
α	0.3	Рекомендованный диапазон 0.1–0.3
Начальные веса	случайные из $[-0.5, 0.5]$, фиксированный seed	Одинаковые веса дают одинаковые нейроны; seed делает результат воспроизводимым
Структура	задаётся параметром: 49-<скрытые слои>-3	Требование: число входов, слоёв и нейронов задаётся параметрами

3. Реализация

Код разделён на три модуля: `net.cpp` (сеть и обучение), `data.cpp` (чтение выборок из текстовых файлов), `main.cpp` (эксперименты и вывод). Ниже функции из `net.cpp`; индекс k : номер слоя, $k = 0$ это входы.

3.1. Прямой проход

Слой за слоем от входа к выходу:

$$y_i^k = f\left(\sum_j y_j^{k-1} w_{ij}^k\right)$$

```
// y_i^k = f( sum_j y_j^(k-1) * w_ij^k ), y^0 = input
const std::vector<double> &Network::forward(const std::vector<double> &input) {
    y[0] = input;
    for (size_t k = 1; k < n.size(); ++k)
        for (int i = 0; i < n[k]; ++i) {
            double sum = 0.0;
            for (int j = 0; j < n[k - 1]; ++j)
                sum += y[k - 1][j] * w[k][i][j];
            y[k][i] = sigmoid(sum);
        }
    return y.back();
}
```

3.2. Ошибка

Для последнего прямого прохода:

$$E = \frac{1}{2} \sum_i (t_i - y_i^N)^2$$

```
// E = 1/2 * sum_i (t_i - y_i^N)^2, for the last forward() call
double Network::error(const std::vector<double> &target) const {
    double e = 0.0;
    for (int i = 0; i < n.back(); ++i) {
        double diff = target[i] - y.back()[i];
        e += diff * diff;
    }
    return 0.5 * e;
}
```

3.3. Обратный проход

Три цикла: δ выходного слоя, δ скрытых слоёв от конца к началу, подстройка весов. Веса меняются после расчёта всех δ , так как формула δ использует веса до подстройки:

$$\Delta w_{ij}^k = \alpha \delta_i^k y_j^{k-1}$$

```
// Backpropagation for the last forward() call.
void Network::backward(const std::vector<double> &target, double alpha) {
    const size_t N = n.size() - 1;

    // Output layer: delta_i^N = y_i^N (1 - y_i^N) (t_i - y_i^N)
    for (int i = 0; i < n[N]; ++i)
        delta[N][i] = y[N][i] * (1.0 - y[N][i]) * (target[i] - y[N][i]);

    // Hidden layers: delta_i^k = y_i^k (1 - y_i^k) sum_j delta_j^(k+1) w_ji^(k+1)
    for (size_t k = N - 1; k >= 1; --k)
        for (int i = 0; i < n[k]; ++i) {
            double sum = 0.0;
            for (int j = 0; j < n[k + 1]; ++j)
                sum += delta[k + 1][j] * w[k + 1][j][i];
            delta[k][i] = y[k][i] * (1.0 - y[k][i]) * sum;
        }

    // All deltas above used the old weights, so update only now:
    // w_ij^k += alpha * delta_i^k * y_j^(k-1)
    for (size_t k = 1; k <= N; ++k)
        for (int i = 0; i < n[k]; ++i)
            for (int j = 0; j < n[k - 1]; ++j)
                w[k][i][j] += alpha * delta[k][i] * y[k - 1][j];
}
```

3.4. Обучение

Эпоха: проход по всей обучающей выборке; подстройка весов на изображениях с $E > \text{eps}$; остановка, когда за эпоху не было ни одной подстройки:

```
// One epoch = one pass over all samples. A sample whose error is above eps
// triggers a weight update. Training stops when a whole epoch passes with
// every sample at E <= eps.
TrainResult Network::train(const std::vector<Sample> &set, double alpha,
                           double eps, int maxEpochs) {
    for (int epoch = 1; epoch <= maxEpochs; ++epoch) {
        bool allLearned = true;
        for (const Sample &s : set) {
            std::vector<double> t = targetFor(s.label);
            forward(s.pixels);
            if (error(t) > eps) {
                backward(t, alpha);
                allLearned = false;
            }
        }
        if (allLearned)
            return {epoch, true};
    }
    return {maxEpochs, false};
}
```

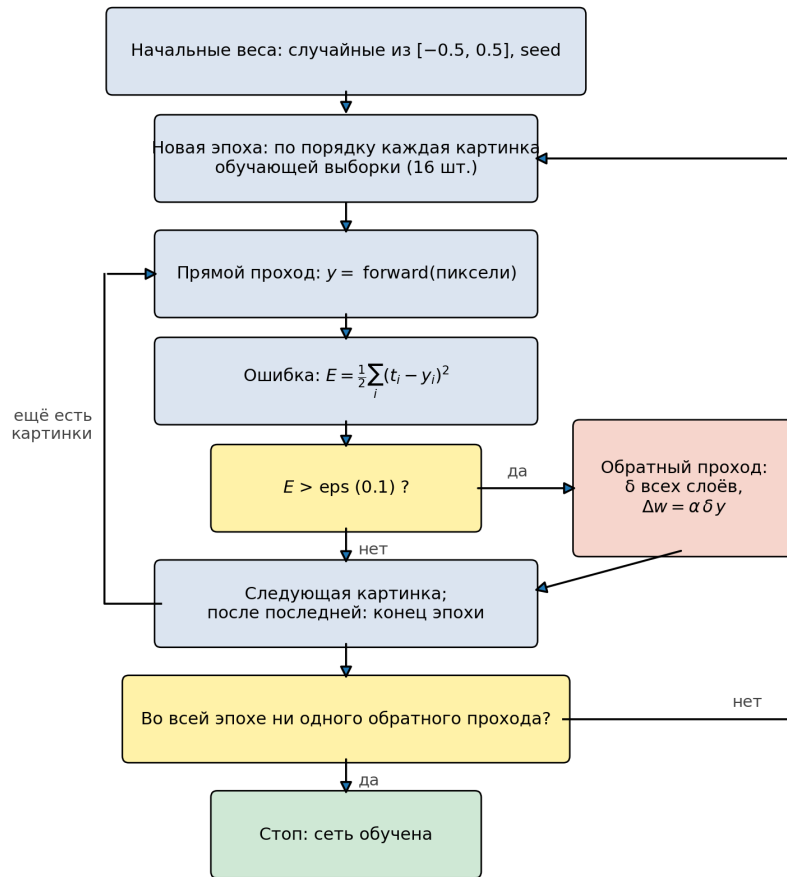


Рисунок 2. Алгоритм обучения

3.5. Классификация

Ответ: выход с наибольшим значением; если он меньше 0.5, изображение считается шумом:

```

// Index of the largest output, or NOISE if even that one is below the
// threshold (no output says "this is my figure").
int Network::classify(const std::vector<double> &input) {
    const std::vector<double> &out = forward(input);
    int best = (int)(std::max_element(out.begin(), out.end()) - out.begin());
    return out[best] < REJECT_THRESHOLD ? NOISE : best;
}
  
```

4. Выборки

Обучающая выборка: эталоны рисунка 1 задания и по одному искажению каждого типа (пропавший пиксель, лишний пиксель, сдвиг на 1 пиксель), четыре вида шума; по 4 изображения на класс. Тестовая выборка: изображения рисунка 2 задания, искажения на 1–3 пикселя, сдвиги в направлениях, которых нет в обучающей выборке, новый шум; с обучающей выборкой не пересекается. Цвет: класс изображения.

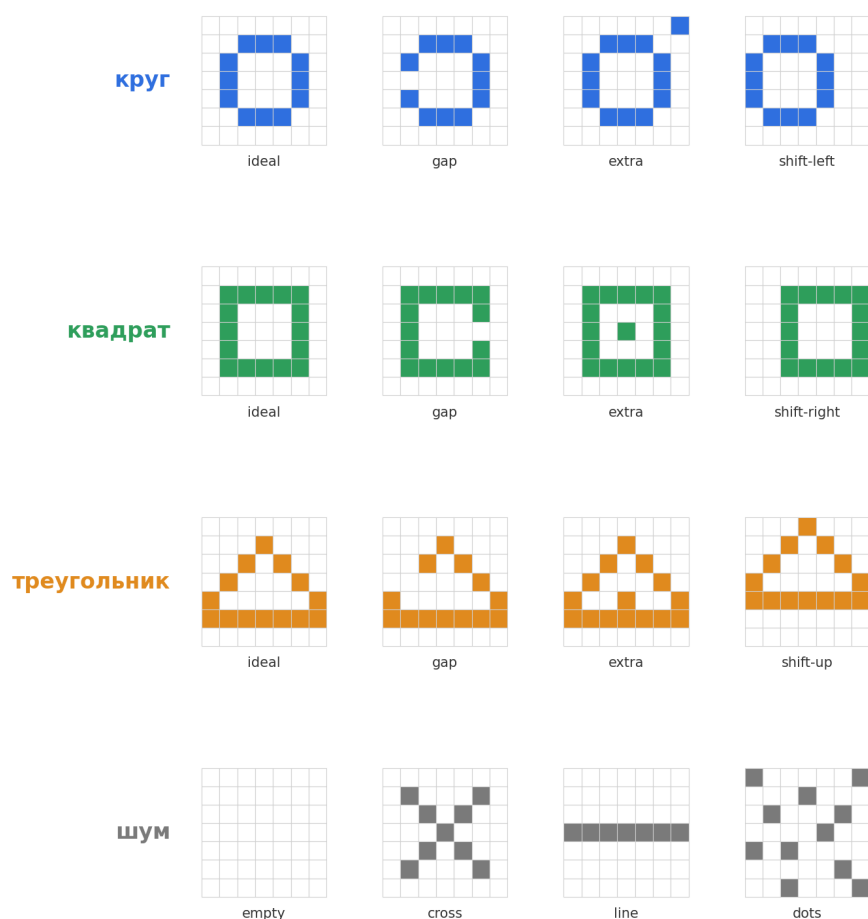


Рисунок 3. Обучающая выборка: 16 изображений, по 4 на класс

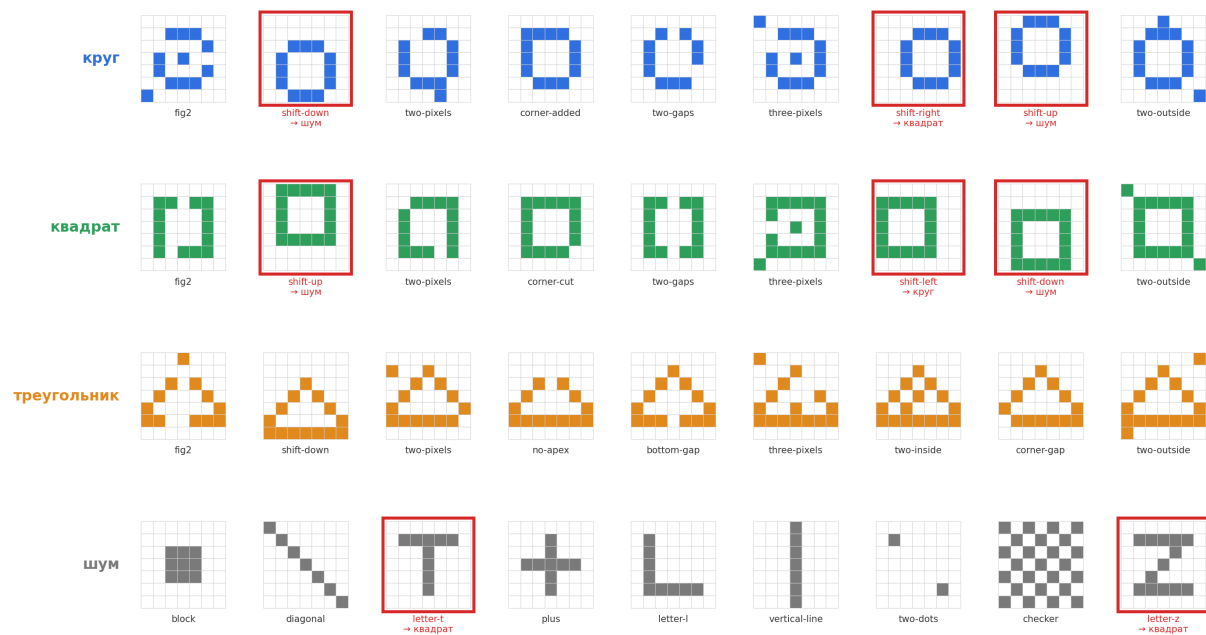


Рисунок 4. Тестовая выборка: 36 изображений, ответы сети 49-8-3 (seed 1); красная рамка: ошибка, под ней ответ сети

5. Результаты

5.1. Сравнение структур

Таблица 1. Сравнение структур: $\alpha = 0.3$, $\text{eps} = 0.1$, среднее по 10 запускам с разными seed. Train и Test: число верно распознанных изображений

Структура	Весов	Сошлось	Эпох	Обучение, мс	Train	Test (среднее)	Test (мин..макс)
49-4-3	208	10/10	64.9	0.61	16/16	25.9/36	24..28
49-8-3	416	10/10	44.3	0.62	16/16	27.2/36	25..28
49-16-3	832	10/10	33.4	0.84	16/16	26.7/36	25..28
49-32-3	1664	10/10	24.9	1.19	16/16	27.1/36	25..29
49-8-8-3	480	10/10	113.0	2.09	16/16	26.5/36	24..28
49-16-8-3	936	10/10	89.9	2.87	16/16	25.8/36	25..27

5.2. Варьирование α

Таблица 2. Сеть 49-8-3, $\text{eps} = 0.1$, среднее по 10 запускам

α	Сошлось	Эпох	Обучение, мс	Test (среднее)	Test (мин..макс)
0.01	10/10	1212.2	17.54	27.0/36	25..28
0.05	10/10	245.2	3.47	26.9/36	25..28
0.10	10/10	124.4	1.84	26.9/36	25..28
0.20	10/10	64.5	0.92	27.2/36	25..28
0.30	10/10	44.3	0.62	27.2/36	25..28
0.50	10/10	28.2	0.39	26.8/36	25..28
0.80	10/10	20.0	0.28	26.9/36	25..28
1.00	10/10	17.6	0.24	26.5/36	25..28

5.3. Длительность обучения

Таблица 3. Сеть 49-8-3, $\alpha = 0.3$; меньший eps означает более долгое обучение

eps	Эпох	Обучение, мс	Test (среднее)	Test (мин..макс)
0.1	44.3	0.63	27.2/36	25..28
0.05	61.1	0.92	26.7/36	25..28
0.01	189.9	2.52	26.2/36	25..28
0.001	1598.1	19.62	25.8/36	25..27

5.4. Характеристики выбранной сети

Таблица 4. Сеть 49-8-3, $\alpha = 0.3$, $\epsilon = 0.1$, seed 1

Характеристика	Значение
Структура	49-8-3 (416 весов)
Эпох обучения	58
Время обучения	0.85 мс
Время вычисления выхода	0.603 мкс
Память: вычисление выхода	3808 Б
Память: обучение	3920 Б
Память: обучающая выборка	6272 Б
Верно на тесте	28/36

Время вычисления выхода: среднее по 100 000 прямым проходам. Память считается по числу хранимых чисел double (8 Б): для вычисления выхода веса и выходы всех нейронов ($416 + 60$), для обучения дополнительно δ нейронов и вектор цели ($11 + 3$).

5.5. Результат на тесте по группам

Таблица 5. Сеть 49-8-3, seed 1

Группа тестовых картинок	Всего	Распознано
Локальные искажения 1-3 пикселя (вкл. рис. 2)	20	20
Сдвиги на 1 пиксель	7	1
Шум	9	7

6. Пример работы сети

Корректно распознаются все искажения на 1–3 пикселя, в том числе изображения рисунка 2 задания (рисунок 4). Сбой: сдвиги, которых нет в обучающей выборке, и шум с горизонтальными отрезками, как у стороны квадрата. Вывод make run для ошибочных изображений (под названием ответ сети):

circle shift-down -> noise###.. .#...#. .#...#. .#...#. .#...#. .#...#. ..###..	square shift-up -> noise .#####. .#...#. .#...#. .#...#. .#...#. .#####.	noise letter-t -> square#####. ...#... ...#... ...#... ...#... ...#...	circle shift-right -> square###. ..#...# ..#...# ..#...# ..#...# ...###.
circle shift-up -> noise ..##### .#...#. .#...#. .#...#. ..#####	square shift-left -> circle #####. #...#.. #...#.. #...#.. #...#.. #####.	square shift-down -> noise#####. .#...#. .#...#. .#...#. .#...#. .#####.	noise letter-z -> square#####.#.. ...#... ..#... ..#... .#####.

7. Выводы

1. Выбрана сеть 49-8-3: 416 весов; 44 эпохи обучения в среднем (58 эпох и 0.85 мс при seed 1); вычисление выхода 0.6 мкс; память 3808 Б на вычисление выхода и 3920 Б на обучение.
2. Сети с 8, 16 и 32 скрытыми нейронами обобщают одинаково (27.2, 26.7 и 27.1 из 36 в среднем); 49-8-3 выбрана как наименьшая из них. 4 нейрона дают худшую точность (25.9) и больший разброс.
3. Второй скрытый слой не улучшает точность и требует в 2–3 раза больше эпох: множитель $y(1 - y) \leq 0.25$ ослабляет ошибку на каждом слое при обратном проходе.
4. α от 0.01 до 1.0 меняет число эпох обратно пропорционально (от 1212 до 18), точность не меняется (26.5–27.2). Уменьшение eps с 0.1 до 0.001 увеличивает число эпох в 36 раз и снижает точность на тесте с 27.2 до 25.8: переобучение.
5. Все искажения на 1–3 пикселя, включая рисунок 2 задания, распознаются (20 из 20).
6. Сдвиги, отсутствующие в обучающей выборке, не распознаются (1 из 7): полносвязная сеть не учитывает соседство пикселей. Шум с элементами фигур (буквы Т и Z) ошибочно распознаётся как квадрат (7 из 9 верно).